

ASSIGNMENT 4

EduSupport

Student Support & Ticket Management Platform

Edumerge Pre-Drive Product Engineering Assignment

Repository: edusupport-ticket-management

Contents

1. Executive Summary
2. Problem Understanding
3. Product Vision
4. Target Users and Roles
5. Functional Requirements
6. User Journeys
 - Student
 - Staff
 - Manager
7. Ticket Lifecycle
8. SLA and Escalation Model
9. Key Product Features
10. Role-Based Access Control
11. System Architecture
12. Database and Domain Model
13. API Design
14. UI and UX Approach
15. Engineering Decisions
16. Assumptions
17. Trade-offs
18. Edge Cases and Failure Scenarios
19. Testing and Validation
20. Security and Data-Safety Considerations
21. Current Limitations
22. Future Improvements
23. AI Usage Report
24. Repository and Run Instructions
25. Conclusion

EduSupport is a college help desk. Students raise requests. Staff work the tickets assigned to them. A manager sees the whole desk, assigns work, and can run the full lifecycle. Operational status and SLA state are separate. Anything not listed as implemented is a limitation or a future improvement.

1. Executive Summary

Students, staff, and managers each have a signed-in workspace. The server enforces who can see a ticket and which status change is legal. A missed deadline can mark a ticket `BREACHED` without changing its operational status. Pending time still counts toward the deadline. Escalation is a manual action. There is no background job that watches the clock, reassigns a ticket, or sends a notification.

The client is React. The API is Express on MongoDB. Hiding a control in the interface is not the permission check.

The server test suite contains 39 passing tests. The server build, server lint, seed, client build, and client lint also passed. Browser and end-to-end testing was not performed. Playwright and Vitest are not configured.

2. Problem Understanding

Colleges receive student requests about fees, attendance, ID cards, certificates, examinations, hostel matters, and technical issues. Without one desk, those requests are hard to assign, prioritize, track, and close.

The assignment asks for statuses, priorities, assignment, SLAs, ageing, ownership, resolution tracking, activity history, management visibility, escalation, and a pending-action path. EduSupport is that desk for one college.

3. Product Vision

One help desk with a fixed status workflow, a visible SLA, an activity history, and three roles. The useful product questions are who may act, which status changes are legal, and how the SLA clock stays independent of operational status.

Email, SMS, attachments, charts, live updates, and automatic operations are outside this prototype.

4. Target Users and Roles

There are three login roles: `student`, `staff`, and `manager`.

Role	Responsibility in this prototype
Student	Create a ticket, follow only their own tickets, add public comments, read the resolution, close a resolved ticket, and reopen a closed ticket with a reason.
Staff	Work only tickets assigned to them. Change status along the allowed transitions, change priority, add public comments and internal notes, resolve, refresh SLA, and escalate manually.

Role	Responsibility in this prototype
Manager	See every ticket. Assign and reassign active staff, change status and priority, comment, resolve, close, reopen, refresh SLA, escalate, and resolve an escalation.

Public registration creates students only. Staff and manager accounts come from the local seed script.

5. Functional Requirements

The prototype includes:

- Student registration, login, and session restore with `GET /api/v1/auth/me`
- Server-side role checks and ticket visibility checks
- Ticket creation with a category, subject, and description
- Ticket numbers from an atomic counter, starting at `EDU-1001`. A failed insert can leave a gap. Gaps are acceptable.
- Manager assignment and reassignment to active staff
- Status changes limited to the state machine
- Priority changes that recalculate the SLA deadline from the original `createdAt`
- SLA statuses `WITHIN_SLA`, `APPROACHING_SLA`, and `BREACHED`, updated by an explicit refresh
- Manual escalation to level 1, then level 2, and resolution of an open escalation
- Public comments and internal notes
- Resolve, close, and reopen as separate actions
- Activity history
- Search and filters
- Role dashboard summaries
- A read-only category list
- A manager-only directory of active staff
- Loading, empty, and error states on the main screens
- Server-side request validation

Section 21 lists what is intentionally not included.

6. User Journeys

Student

Sign in, or register and then sign in. Registration does not return a token. The student home reads `GET /api/v1/dashboard/student`. The student creates a ticket from an active category. The server sets the owner, number, default priority, SLA policy, and deadline. The student follows that ticket, adds public comments, reads the activity timeline and the stored SLA, closes it after it is resolved, and can reopen it after it is closed.

Staff

The staff home reads `GET /api/v1/dashboard/staff` and counts only assigned tickets. The staff member opens an assigned ticket, moves it through the allowed statuses, adds a public reply or an internal note, changes priority, writes a resolution from `IN_PROGRESS`, refreshes the SLA, and escalates manually. Close, reopen, and assignment are not available to staff.

Manager

The manager home reads `GET /api/v1/dashboard/manager` for the whole organization, including category counts and staff workload. The manager filters every ticket, assigns or reassigns active staff, and can run the lifecycle, comments, SLA refresh, and escalations. Managers cannot create tickets. `POST /api/v1/tickets` remains student-only.

7. Ticket Lifecycle

OPEN → ASSIGNED → IN_PROGRESS → PENDING → IN_PROGRESS → RESOLVED → CLOSED

A closed ticket can be reopened to `IN_PROGRESS`. Reopen does not extend the SLA deadline.

`PATCH /api/v1/tickets/:ticketId/status` accepts only these pairs:

From	To
OPEN	ASSIGNED
ASSIGNED	IN_PROGRESS
IN_PROGRESS	PENDING
PENDING	IN_PROGRESS

`OPEN` to `ASSIGNED` on that route requires an assignee already set. The normal first assignment is `PATCH /api/v1/tickets/:ticketId/assignment`, which moves an `OPEN` ticket to `ASSIGNED`. Reassignment changes the assignee only. Status, `createdAt`, the SLA deadline, and the escalation level stay as they are.

Resolve, close, and reopen have their own routes. The generic status route cannot complete them.

Action	Rule
Resolve	Staff on an assigned ticket, or a manager. Status must be <code>IN_PROGRESS</code> . Resolution text is required. The server sets <code>resolution</code> , <code>resolvedAt</code> , and <code>RESOLVED</code> .
Close	Owning student or a manager. Status must be <code>RESOLVED</code> . The server sets <code>closedAt</code> and <code>CLOSED</code> . Staff cannot close.
Reopen	Owning student or a manager. Status must be <code>CLOSED</code> . A reason is required. Status returns to <code>IN_PROGRESS</code> . Stored resolution and resolution timestamps are cleared. The previous resolution is kept on the <code>REOPENED</code> activity. There is no time limit.

Using the generic status route to resolve returns `422 USE_RESOLVE` when a resolution is already stored, or `400` when it is not. Using it to close returns `422 USE_CLOSE`. Any other illegal pair returns `422 INVALID_TRANSITION`.

Pending does not pause the SLA and does not require a comment.

8. SLA and Escalation Model

These durations are product assumptions. The assignment did not prescribe them.

Priority	Resolution window
CRITICAL	4 hours
HIGH	8 hours
MEDIUM	24 hours
LOW	48 hours

```
slaDeadline = createdAt + resolutionTimeHours
```

The clock is wall-clock time from creation. It is not limited to office hours. Approaching means 25 percent or less of the original window remains and the deadline is still ahead.

Seeded categories use default priority `MEDIUM`. Each SLA policy stores the same number of hours for response time and resolution time. The deadline uses resolution hours.

The client cannot set `slaStatus` or `slaDeadline`. A new ticket starts as `WITHIN_SLA`. A priority change recalculates the deadline from the original `createdAt` and recalculates `slaStatus`. It does not change operational status or `escalationLevel`.

`PATCH /api/v1/tickets/:ticketId/sla/refresh` recalculates `slaStatus` from the stored deadline. The new status is saved only when it changes. Moving to `BREACHED` records one `SLA_BREACHED` activity. A second refresh does not record another. Operational status does not change, and the ticket is not reassigned.

`GET /api/v1/tickets/sla-summary` counts stored values for a manager, or for the caller's assigned tickets if the caller is staff. It does not write a new SLA status. Students receive `403`.

Overdue means a non-`CLOSED` ticket whose `slaDeadline` is at or before now. Dashboard reads do not rewrite SLA state and do not write activities.

Manual escalation:

Rule	Behavior
First open escalation	<code>LEVEL_1</code>
Second open escalation	<code>LEVEL_2</code>
Another open escalation	<code>409 ESCALATION_ALREADY_OPEN</code>
Who may escalate	Assigned staff or a manager
Server-owned fields	Level, <code>triggeredBy</code> , and assignee fields
Ticket side effects	Assignee, status, and SLA deadline stay unchanged. <code>newAssignee</code> stays null.
Resolve an escalation	Sets that escalation to <code>RESOLVED</code> . Ticket status and the SLA deadline do not change. Resolving it again is <code>409 ESCALATION_NOT_OPEN</code> .

There is no background SLA job and no automatic escalation.

9. Key Product Features

- Three workspaces with role navigation and shared ticket badges
- Student create, list, detail, public comments, activity, close, and reopen
- Staff assigned queue, status changes, priority, public and internal comments, resolve, SLA refresh, escalation, and ticket age
- Manager organization list, assignment, lifecycle, comments, SLA refresh, and escalations
- SLA badges show the stored status: Within SLA, Approaching SLA, or Breached
- Ticket age on staff and manager screens is calculated in the browser from `createdAt`. It is not an authoritative SLA status.
- Internal notes stay in their own panel. The activity line says an internal note was added and does not repeat the note text.
- Dashboards are counts and recent tickets, not charts

10. Role-Based Access Control

Every private API route checks the caller. React route guards only redirect. A student who calls a staff or manager API still receives 403.

Action	Student	Staff	Manager
Create a ticket	Yes	No	No
List and read tickets	Own tickets only	Assigned tickets only	All tickets
Assign or reassign	No	No	Yes, to active staff
Generic status change	No	Assigned ticket, allowed transitions only	Any ticket, same transitions
Change priority	No	Assigned ticket, unless resolved or closed	Any ticket, unless resolved or closed
Public comment	Own ticket	Assigned ticket	Any ticket
Internal note	No	Assigned ticket	Any ticket
Resolve	No	Assigned ticket, from IN_PROGRESS	Any ticket, from IN_PROGRESS
Close	Own ticket, from RESOLVED	No	Any ticket, from RESOLVED
Reopen	Own ticket, from CLOSED	No	Any ticket, from CLOSED
Refresh SLA	No	Assigned ticket	Any ticket
SLA summary	No	Assigned tickets	Organization
Escalate and resolve an escalation	No	Assigned ticket	Any ticket
Staff directory	No	No	Yes
Category list	Active categories	Active categories	Active and inactive categories

Action	Student	Staff	Manager
Dashboards	Own tickets	Assigned tickets	Organization

A ticket the caller cannot see returns `404 TICKET_NOT_FOUND`, the same response as a missing ticket. An invalid id is `400`.

11. System Architecture

```

Browser (React, Vite, TypeScript, Tailwind, React Router)
  |
  ▼
Express API /api/v1
  | authentication, role check, service, Mongoose
  ▼
MongoDB

```

The repository is an npm workspace with `client/` and `server/`. Docker Compose can start MongoDB 7 for local development.

The local database is a standalone MongoDB server. A ticket write and its activity write are sequential, not a multi-document transaction. If the activity insert fails, the service restores or removes the earlier write.

Success responses use `{ "success": true, "data": { } }`. Errors use `{ "success": false, "error": { "code", "message" } }`. Unexpected failures return `500` with the message "Something went wrong." and do not return a stack trace. Invalid JSON is `400 INVALID_JSON`.

`GET /api/v1/health` reports that the API is running and whether the database is connected. It does not return the connection string.

12. Database and Domain Model

Database name: `edusupport`.

Collection	What it stores
<code>users</code>	Name, email, <code>passwordHash</code> (not selected by default), role, optional department, <code>isActive</code>
<code>categories</code>	Name, description, <code>defaultPriority</code> , <code>isActive</code>
<code>slapolicies</code>	Priority, <code>responseTimeHours</code> , <code>resolutionTimeHours</code> , <code>isActive</code>
<code>tickets</code>	Number, student, assignee, category, subject, description, priority, status, SLA policy, deadline, SLA status, escalation level, resolution, timestamps
<code>comments</code>	Ticket, author, message, type <code>PUBLIC</code> or <code>INTERNAL</code>
<code>activities</code>	Ticket, actor, action, old value, new value, metadata, timestamp
<code>escalations</code>	Ticket, level, status, reason, who triggered it, assignee fields, timestamps
<code>counters</code>	Atomic sequence used to allocate ticket numbers

Password hashes are not returned by the API. There is no notifications collection. Tickets do not store a concurrency version.

13. API Design

Base path: /api/v1.

```

POST  /api/v1/auth/register
POST  /api/v1/auth/login
GET   /api/v1/auth/me

POST  /api/v1/tickets
GET   /api/v1/tickets
GET   /api/v1/tickets/:ticketId

PATCH /api/v1/tickets/:ticketId/assignment
PATCH /api/v1/tickets/:ticketId/status
PATCH /api/v1/tickets/:ticketId/priority

GET   /api/v1/tickets/:ticketId/comments
POST  /api/v1/tickets/:ticketId/comments

POST  /api/v1/tickets/:ticketId/resolve
POST  /api/v1/tickets/:ticketId/close
POST  /api/v1/tickets/:ticketId/reopen
POST  /api/v1/tickets/:ticketId/escalate
GET   /api/v1/tickets/:ticketId/escalations
POST  /api/v1/tickets/:ticketId/escalations/:escalationId/resolve

PATCH /api/v1/tickets/:ticketId/sla/refresh
GET   /api/v1/tickets/sla-summary

GET   /api/v1/tickets/:ticketId/activities

GET   /api/v1/dashboard/student
GET   /api/v1/dashboard/staff
GET   /api/v1/dashboard/manager

GET   /api/v1/categories
GET   /api/v1/users/staff

GET   /api/v1/health

```

Category create and update, and SLA-policy read and update, are not part of this API.

Create accepts only `categoryId`, `subject`, and `description`. The server sets the student, ticket number, priority, SLA policy, deadline, SLA status, status `OPEN`, and escalation level. A client-supplied owner, status, priority, SLA field, or ticket number is rejected.

Comments take `message` and `type`. `authorId` comes from the token. Students can send only `PUBLIC`. A student comment list stays public even if the query asks for internal notes. Student activity lists omit `COMMENT_ADDED` rows whose value is `INTERNAL`.

Assignment accepts only `assignedTo`. The target must be an active staff user. Resolved and closed tickets cannot be assigned. An assignee cannot be cleared.

List filters include `page`, `limit`, `status`, `priority`, `category`, `assignee`, `SLA status`, `overdue`, `search`, and `sort`. Search matches subject and ticket number. A student query cannot widen the list with `assignedTo`. A staff

query cannot widen it beyond that staff member's assignments.

Dashboard	Scope and shape
Student	Own tickets. Counts, priority counts, SLA counts without overdueOpen, and up to 5 recent tickets.
Staff	Assigned tickets. assigned means status ASSIGNED, plus in progress, pending, resolved, closed, overdueOpen, escalation counts, and up to 5 recent tickets.
Manager	Organization-wide counts, category counts, and workload for active staff who have at least one assigned ticket. In that workload, assigned means every ticket assigned to that person. Up to 10 recent tickets.

Login returns the token and the public user inside data. An unknown email, a wrong password, and an inactive account share one 401: "Invalid email or password."

14. UI and UX Approach

The shell shows the EduSupport mark, a role subtitle, navigation for that role only, the user's name, a role badge, and log out. On a narrow screen the navigation stacks.

Path	Who
/login, /register	Signed-out visitors
/student, /student/tickets, /student/tickets/new, /student/tickets/:id	Students
/staff, /staff/tickets, /staff/tickets/:id	Staff
/manager, /manager/tickets, /manager/tickets/:id	Managers

An unknown path sends a signed-out visitor to login and a signed-in user to their role home. The wrong role is sent to that user's home.

Main screens have a loading state, an API error, and an empty state. Submit actions disable the control while the request is in flight.

A signed-in request that is no longer authorized says the session has expired. A wrong password keeps the invalid-credentials message.

The access token is stored in localStorage under edusupport_token.

15. Engineering Decisions

- Resolve, close, reopen, and escalate are separate routes, so one generic patch cannot set status and the SLA deadline together.
- One ticket list serves every role. The server applies the scope.
- SLA status is stored and refreshed explicitly. The client displays that stored value.
- A hidden ticket and a missing ticket both return 404, so one student cannot learn that another student's ticket exists.

- Public registration cannot choose a role.
- Each authenticated request reloads the user. The database role is enforced. An inactive user is rejected.
- Access tokens expire after 8 hours. The payload carries the user id and role.
- Passwords are hashed with bcrypt, 12 salt rounds. `passwordHash` is excluded from normal queries and is not returned.
- Ticket numbers use an atomic counter. The public number is 1000 plus the sequence.
- An internal-note activity stores the comment type, not the note text.

16. Assumptions

- One college. SLA math uses elapsed hours, not business hours. Dates are displayed with `en-IN` formatting.
- The SLA durations and the 25 percent approaching threshold are choices for this prototype.
- Categories and SLA policies are seed data. They are not edited through the API.
- Students see only their own tickets. Staff see assigned tickets, not a whole category. Managers see every ticket.
- Internal notes are for staff and managers.
- There is no mobile app, payment integration, chatbot, or extra third-party service.
- Local seed accounts are for a local database only. The seed script refuses to run when `NODE_ENV` is `production`.

17. Trade-offs

- SLA status is refreshed on demand. It can lag until someone refreshes it. A dashboard read never writes SLA state.
- Escalation does not reassign the ticket. A breach does not move ownership to a manager.
- `PENDING` does not pause the clock. Waiting on the student still consumes the deadline.
- Reopen has no time limit. The original deadline still does not move, so a late reopen can already be breached.
- Writes are sequential because the local MongoDB server is standalone.
- The token is in `localStorage`. A production deployment could use a hardened cookie. This prototype does not.
- Managers get counts, category totals, and a workload table rather than charts.

18. Edge Cases and Failure Scenarios

Covered by the server test suite:

- A student reading another student's ticket receives `404 TICKET_NOT_FOUND`
- A jump from `OPEN` to `RESOLVED` is rejected
- Staff cannot reassign a ticket (`403`)

- Manager reassignment changes the assignee and leaves status, `createdAt`, the deadline, and the escalation level unchanged
- An internal note is omitted from a student's comments and from that student's activity history
- A student can close a resolved ticket
- A student can reopen a closed ticket. The resolution and timestamps are cleared, the old resolution stays on the `REOPENED` activity, and the SLA deadline does not move
- A passed deadline leaves operational status unchanged. Refresh sets `BREACHED`, records one `SLA_BREACHED` activity, and does not assign a manager
- Each dashboard rejects the other roles
- The staff directory omits the password hash and rejects staff and anonymous callers

Also enforced by the API:

- An empty resolution is rejected and the ticket stays unchanged
- Resolve is rejected unless the status is `IN_PROGRESS`
- A third open escalation is rejected
- A student, a manager, an inactive user, or any non-staff user cannot be assigned
- Resolved and closed tickets cannot be assigned, and their priority cannot be changed

Duplicate detection and optimistic concurrency are not part of the API. See section 21.

19. Testing and Validation

Checked on 25 September 2026:

Check	Result
Server test suite	39 passing tests
Server build	Passed
Server lint	Passed
Seed	Passed
Client build	Passed
Client lint	Passed

The server test suite contains 39 passing tests. It covers authentication, the ticket domain, ticket APIs, workflow, lifecycle, SLA and escalation, dashboards, the category list, and the staff directory.

Browser and end-to-end testing was not performed. Playwright and Vitest are not configured. Client build and lint are the frontend checks that were run.

20. Security and Data-Safety Considerations

- `.env` files are gitignored. The example files contain placeholders and a local MongoDB address.
- `JWT_SECRET` is required on the server and is not sent to the client.
- Passwords are hashed. The hash is not selected by default and is not included in API responses.

- The staff directory returns `id`, `name`, and `email` for active staff only.
- The client does not send `authorId`, `studentId`, `slaStatus`, or `slaDeadline`.
- Students cannot choose priority, status, SLA, or assignee when creating a ticket.
- Students do not receive internal comments or internal-note activity rows.
- CORS is limited to `CLIENT_URL`.
- The health check does not reveal the MongoDB URI.
- Login uses one message for an unknown email, a wrong password, and an inactive account.

Login rate limiting is not implemented. The access token remains in `localStorage`.

21. Current Limitations

- No background SLA worker. Stored SLA status changes when a ticket is created, when priority changes, or when someone refreshes it.
- No notifications, email, or SMS.
- No real-time updates.
- No automatic reassignment.
- No charts.
- No category administration and no SLA-policy administration.
- No duplicate-ticket warning.
- No optimistic concurrency.
- No time limit on reopen.
- Staff see assigned tickets only, not every unassigned ticket in a category.
- Managers cannot create tickets, and an assignee cannot be cleared.
- Ticket and activity writes are not transactional.
- The access token is stored in `localStorage`.
- Browser click-through was not performed.
- Playwright and Vitest are not configured.

22. Future Improvements

Not built in this prototype:

- A scheduled refresh that updates SLA status and records a breach once, without changing operational status
- Notifications for assignment, public replies, resolution, and breach
- Optional escalation rules that still leave a clear activity record
- Manager charts
- Category and SLA-policy administration
- Duplicate detection before create
- Optimistic concurrency
- A reopen window

- Attachments, a knowledge base, satisfaction scores, and more than one campus
- End-to-end browser tests
- A hardened cookie for the access token, and rate limiting on login and registration

23. AI Usage Report

AI tool: Cursor with Grok.

AI was used for planning, architecture discussion, implementation assistance, test generation and review, documentation, and UI development. Outputs were reviewed against the assignment and the running application. Incorrect assumptions were corrected during implementation, including an early business-hours clock and automatic reassignment on breach. The implemented SLA is wall-clock time, pending does not pause it, and escalation is manual.

Final validation was the server test suite, server and client builds, linting, seed validation, and a source-level review of security and authorization. Browser validation was not performed.

The dated log is in `docs/AI_USAGE_REPORT.md`.

24. Repository and Run Instructions

Requirements: Node.js 20 or newer, npm, and Docker if Compose should start MongoDB. MongoDB must be reachable at the URI in `server/.env`.

```
npm install
copy server\.env.example server\.env
copy client\.env.example client\.env
docker compose up -d
npm run seed -w server
npm run dev:server
npm run dev:client
```

The API listens on port 4000. The client listens on port 5173. Health check: `GET http://localhost:4000/api/v1/health`.

Set `JWT_SECRET` in `server/.env` before starting the API. Do not commit `.env` files.

Local development accounts, created only by the seed script:

Role	Email	Password
student	student@edusupport.local	Student123
staff	staff@edusupport.local	Staff1234
manager	manager@edusupport.local	Manager123

Command	What it does
<code>npm test -w server</code>	Server test suite
<code>npm run build</code>	Compile the server, then the client
<code>npm run lint</code>	ESLint in both packages

Further notes: `docs/API.md`, `docs/VALIDATION.md`, and `docs/AI_USAGE_REPORT.md`.

25. Conclusion

EduSupport is a three-role help desk with a server-enforced status machine, a separate SLA clock, and manual escalation. Students own their tickets. Staff work what is assigned to them. Managers see the organization and assign the work.

The server test suite contains 39 passing tests. Server build, server lint, seed, client build, and client lint passed. Browser and end-to-end testing was not performed. Charts, notifications, real-time updates, automatic reassignment, background SLA monitoring, and category or SLA-policy administration are not part of this prototype.